

End-to-End CI/CD Pipeline with Helm, ArgoCD, and GitHub Actions

A GitOps-Driven Approach to Cloud-Native Application Delivery

Ayomide Ajayi

Larry.ayomide@techbleat.co.uk

1.0 Project Overview

This project implements the **automated deployment** of a **containerized web application** on a **Kubernetes cluster** using **Helm charts** and **GitOps principles**. A **four-node Kubernetes cluster** was created using **KIND (Kubernetes IN Docker)** on **Docker Desktop** to simulate a production-like environment.

The deployment process involved **building and containerizing the application** using a **Dockerfile**, then **pushing the image to a local container registry** for efficient image storage and access. **Helm charts** were configured to define the application's Kubernetes resources, ensuring a standardized, reproducible, and version-controlled deployment.

To achieve **continuous integration and deployment (CI/CD)**, **GitHub Actions** (with a self-hosted runner) was used to automate the Helm-based deployment pipeline—triggering builds and releases on code changes. **ArgoCD** was then integrated to **manage application deployments declaratively** using **GitOps**. It **continuously monitored the Helm chart repository** and **synchronized** the cluster state with the desired configuration stored in Git, **enabling automated rollbacks, drift detection, and visual insight** into the deployment lifecycle. Finally, **port-forwarding** was used to enable local access to the deployed application, providing interactive access to the running service within the cluster.

This project demonstrates a **scalable, automated, and GitOps-driven approach** to **deploying containerized applications in Kubernetes**—leveraging **Helm for Kubernetes packaging**, **GitHub Actions for CI/CD orchestration**, and **ArgoCD for declarative and auditable GitOps delivery**.

2.0 Technologies Used

Docker

Docker is a **containerization platform** that enables applications to be packaged into **lightweight, portable containers**. These containers encapsulate all dependencies, ensuring consistency across different environments. Docker allows services to run in **isolated environments**, either on the same host or distributed across multiple hosts, improving scalability and resource efficiency.

Kubernetes

Kubernetes is an **open-source container orchestration platform** designed to **deploy, manage, and scale** containerized applications. It automates key processes such as **service discovery, load balancing, and self-healing** while ensuring high availability and fault tolerance. Kubernetes also supports **auto-scaling**, enabling applications to dynamically adjust resources based on demand.

Helm

Helm is a **package manager for Kubernetes** that simplifies application deployment by bundling multiple Kubernetes resource files into a **single package**, known as a **Helm chart**. Helm allows applications to be installed with a **single command**, providing a default configuration while enabling customization through **values files**. This streamlines deployment, version control, and rollback processes, making Kubernetes application management more efficient.

3.0 Why Use Helm Charts?

Using **Helm charts** for deploying a web application on Kubernetes is far more **scalable, reusable, and maintainable** than using raw manifest files. The following presents the advantages of using Helm Charts over raw manifest files:

1. Parameterization & Configurability

- Helm allows you to define **values.yaml** files, making deployments **customizable** without modifying core Kubernetes manifests.
- Raw YAML files require manual edits, leading to inconsistent configurations across environments.

2. Versioning & Rollbacks

- Helm tracks releases, making it easy to **rollback** to a previous state:
e.g `helm rollback coweb 2`
- However, with raw manifests, reverting requires manually undoing changes or maintaining backups.

3. Dependency Management

- Helm **bundles dependencies** into charts (charts/ directory), handling things like database deployments within the same package.
- With raw manifests, you need to manage external dependencies separately.

4. Simplified Deployment with One Command

- Instead of applying multiple YAML files manually, Helm deploys everything in one go: `helm upgrade --install coweb ./coweb --namespace coweb-ns`
- Unlike raw manifests that require multiple `kubectl` apply commands, increasing complexity.

5. Dry-Run & Pre-Validation

- Helm supports **template rendering & dry run**:
`helm template ./coweb`
`helm upgrade --install coweb ./coweb --dry-run --debug`
- Raw manifests lack built-in validation beyond Kubernetes' basic syntactic checks.

6. Built-in Templating & Reusability

- Helm templates enable reuse across different and multiple environments (dev, staging, prod) without file duplications.
- Raw manifests require multiple static YAML files for each environment, increasing maintenance effort.

When to Use Raw Manifests Instead

- For small, **single-use, manual deployments**.
- When dealing with **custom CRDs** not easily managed via Helm.

In summary, Helm streamlines deployments, reduces human error, and efficiently scales Kubernetes applications. Its ability to ensure repeatability and flexibility makes it a practical solution for managing production-grade web applications with reliability and ease.

4.0 Project Implementation

4.1 Prerequisites

- **Docker Desktop** installed and running.
- **Kubernetes enabled** within Docker Desktop.
- **Helm installed** on your local machine.
- **GitHub repository** with the application source code.
- **Self-hosted GitHub Actions runner** configured on the local machine.
- **ArgoCD manually configured (service exposed)** on the **local K8S cluster**
- **Basic knowledge** of Docker, Kubernetes, Helm, and GitHub Actions.

4.2 Step-by-Step Implementation

Follow this **structured approach** to implement **CI/CD** for deploying the **containerized web application** on a **Kubernetes cluster running on Docker Desktop**, using **Helm and GitHub Actions**. This workflow ensures **continuous integration and deployment (CI/CD)** by automating the **build, testing, and deployment** process using dynamic and custom-built **Kubernetes manifests**, enabling a streamlined and repeatable deployment pipeline.

Step 1: Containerize and Push Application

- **Create a Dockerfile** for the web application:
- **Build the Docker image:** `docker build -t coweb .`
- **Push the image to Docker Hub:** After tagging

Step 2: Create a Helm Chart

- Run the following command to generate a new Helm chart structure:

`helm create coweb`

This creates a directory named `coweb` that contains default templates for deployments, services, and configurations.

Step 3: Define the Deployment

- Modify the Helm **Deployment template** (`coweb/templates/deployment.yaml`)
- Uses Helm template variables (`{{ .Values.image.repository }}`) to dynamically set the **Docker image**.
- Ensures **six replicas** of the web application are running.

Step 4: Define the Service

- Modify the Helm **Service template** (coweb/templates/service.yaml)
- Exposes the web application on **port 80** using Kubernetes **NodePort**.

Step 5: Configure Helm Values

- Modify the default values in my-web-app/values.yaml
- This ensures Helm dynamically sets the **Docker image repository and tag**.

Step 6: Deploy the Helm Chart

- Run the following command to install the Helm chart:
`helm install coweb ./coweb --namespace coweb-ns`
- Verify that the deployment and service are running:
`kubectl get pods -n coweb-ns`
`kubectl get svc -n coweb-ns`

Step 7: Automate CI/CD with GitHub Actions

- **Configure the Self-Hosted Runner and start the runner:** `./rum.cmd` or `./run.sh`

Step 8: Create GitHub Actions Workflow

- **Create a .github/workflows/deploy.yml file** in the repository.
- **Define the workflow**

Step 9: Configure GitHub Secrets: `if NOT` using self-hosted runner

Step 10: Trigger Deployment: `Push changes to the repository`

Step 11: Access the Application

- **Use port-forwarding to access the application:**
`kubectl port-forward svc/coweb-service 31555:80 -n coweb-ns`
- **Open the application in a browser:**
`http://localhost:31555`

4.3 Key Project Highlights

Helm Linting Workflow

The `helm lint ./coweb` command validates the Helm chart in the `coweb` directory, checking `Chart.yaml`, `values.yaml`, and templates for syntax errors, missing values, and adherence to best practices. Helm processes templates to ensure correct references to defined values. If misconfigurations such as undefined `.Values` references, incorrect field types, or structural issues are detected, it generates warnings or errors that may lead to deployment failures. Linting helps maintain Helm charts by ensuring consistency across environments and preventing runtime errors. As a key step in CI/CD pipelines, it catches issues before deployment, reducing risks and improving reliability.

CodeRabbit

CodeRabbit is an AI-powered code review tool that integrates with GitHub to automate pull request reviews. Installed in VS Code and triggered on push, it analyzes code changes, summarizes identified issues, and suggests improvements using AI models. It can also be integrated into CI/CD pipelines by authorizing CodeRabbit on GitHub, configuring it for the repository, and enabling it in GitHub Actions. This provides several benefits, including reducing manual code reviews, AI-powered issue detection, continuous feedback on commits within pull requests, and customizable rules via `.coderabbit.yaml`.

Helm Rollbacks & Kubernetes Health Checks

Helm rollbacks ensure application stability by reverting deployments to previous functional versions in case of failures. Users can restore an earlier release by specifying the release name and revision. Tracking rollback history improves transparency, helping teams understand deployment evolution.

Kubernetes health checks enhance application reliability through liveness and readiness probes. Liveness probes restart unhealthy containers, while readiness probes prevent unready instances from receiving traffic. Configuring these probes in `deployment.yaml` improves uptime and mitigates cascading failures.

Integrating Helm rollbacks with Kubernetes health checks strengthens deployment resilience, minimizing downtime and enhancing fault tolerance. Automating rollback triggers based on failed health checks ensures rapid recovery. Teams can further optimize workflows by incorporating Kubernetes StatefulSets for consistent recovery strategies.

CI/CD Integration with Helm

Integrating Helm chart deployment with CI/CD automation using GitHub Actions streamlines updates. Changes pushed to a repository trigger automatic Helm deployments while validating configurations through linting to catch errors early. In case of deployment failures, Helm's rollback mechanism restores the previous stable version, preventing disruptions. Additionally, Helm's upgrade strategy enables zero-downtime updates, maintaining application availability.

ArgoCD

The deployment of the containerised web application using Helm Kubernetes cluster was also integrated with ArgoCD (which was installed manually on the local machine) to enable GitOps-style continuous delivery once the GitHub Actions workflow is triggered.

By combining **Helm rollbacks, Kubernetes health checks, CI/CD automation, ArgoCD's declarative GitOps synchronization** and **AI-powered code reviews**, deployments become more resilient and self-healing. ArgoCD continuously monitors the desired state stored in Git, ensuring automatic remediation and rollback in the event of discrepancies or failures. This reduces manual intervention while accelerating recovery, making it ideal for production environments where **uptime, stability, and automated deployment integrity** are critical.

5.0 Project Artefacts

Four-Node Kubernetes Cluster in KIND

```
PS C:\Users\ajayi> kubectl get nodes
NAME                                STATUS    ROLES    AGE   VERSION
desktop-control-plane              Ready    control-plane   3d3h   v1.31.1
desktop-worker                     Ready    <none>         3d3h   v1.31.1
desktop-worker2                    Ready    <none>         3d3h   v1.31.1
desktop-worker3                    Ready    <none>         3d3h   v1.31.1
PS C:\Users\ajayi> |
```

Web Application Deployment with 8 Replicas

```
PS C:\Users\ajayi> kubectl get deploy -n coweb-ns
NAME    READY   UP-TO-DATE   AVAILABLE   AGE
coweb   8/8     8            8           10h
PS C:\Users\ajayi> kubectl get pods -n coweb-ns
NAME                                READY   STATUS    RESTARTS   AGE
coweb-699dbfc4bd-5bbkq              1/1     Running   0           10h
coweb-699dbfc4bd-jsc9r              1/1     Running   0           10h
coweb-699dbfc4bd-lrslt              1/1     Running   0           10h
coweb-699dbfc4bd-qgfsd              1/1     Running   0           10h
coweb-699dbfc4bd-rkfm7              1/1     Running   0           10h
coweb-699dbfc4bd-vnkcw              1/1     Running   0           10h
coweb-699dbfc4bd-xhvsx              1/1     Running   0           10h
coweb-699dbfc4bd-zchcw              1/1     Running   0           10h
PS C:\Users\ajayi> |
```

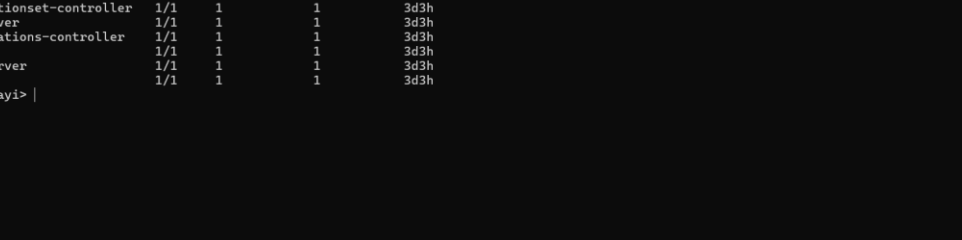
Web Application Service

```
PS C:\Users\ajayi> kubectl get svc -n coweb-ns
NAME          TYPE        CLUSTER-IP    EXTERNAL-IP    PORT(S)    AGE
coweb-service ClusterIP    10.96.83.109   <none>          80/TCP      10h
PS C:\Users\ajayi>
```

SVC Port-Forwarding

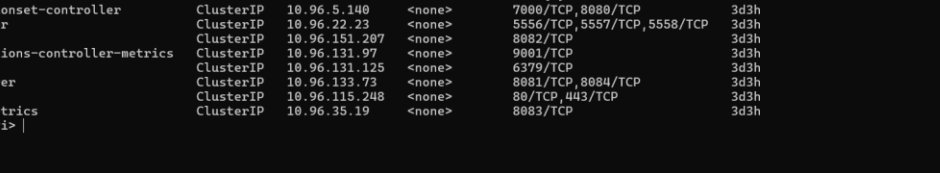
```
PS C:\Users\ajayi> kubectl port-forward svc/coweb-service 31500:80 -n coweb-ns
Forwarding from 127.0.0.1:31500 -> 80
Forwarding from [::1]:31500 -> 80
|
```

ArgoCD Deployment



```
PS C:\Users\ajayi> kubectl get deploy -n argocd
NAME                                READY    UP-TO-DATE    AVAILABLE    AGE
argocd-applicationset-controller    1/1      1              1            3d3h
argocd-dex-server                   1/1      1              1            3d3h
argocd-notifications-controller    1/1      1              1            3d3h
argocd-redis                        1/1      1              1            3d3h
argocd-repo-server                  1/1      1              1            3d3h
```

ArgoCD SVC

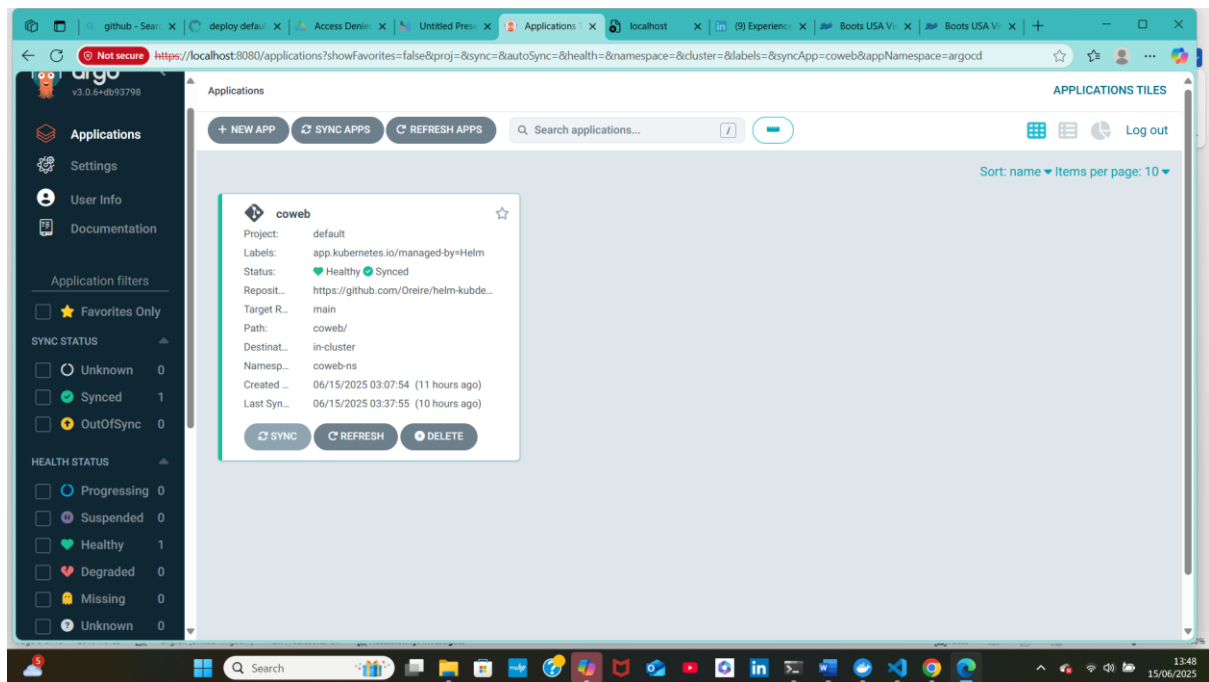


```

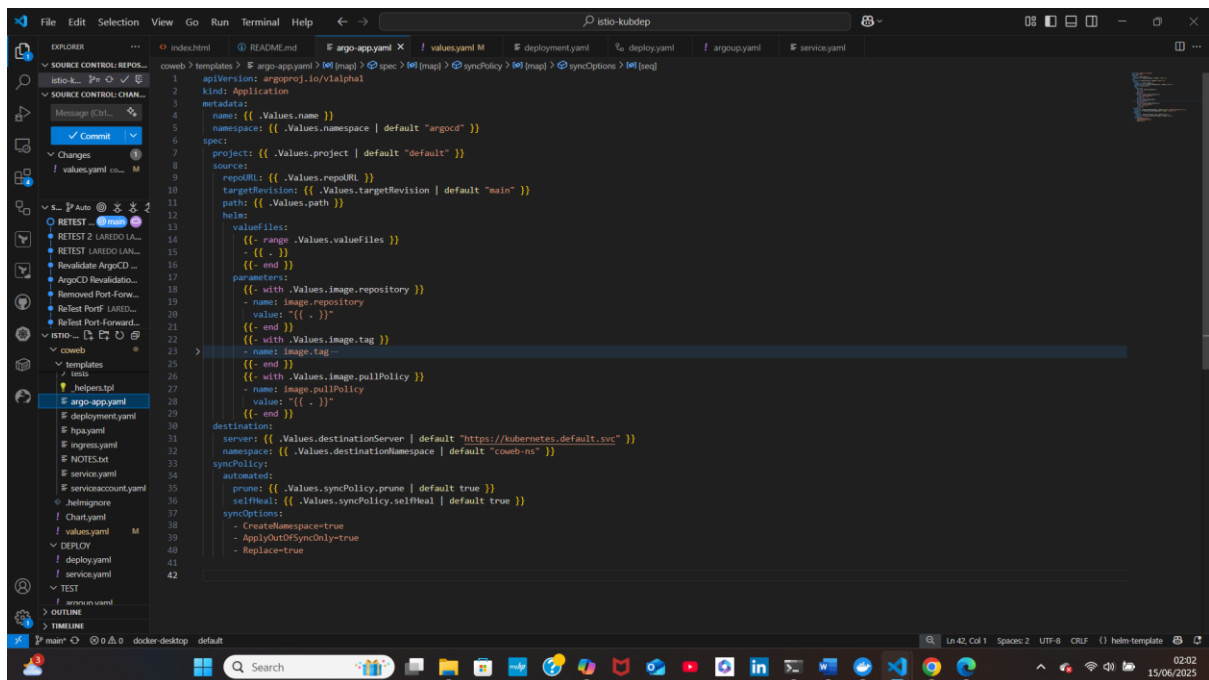
PS C:\Users\ajayi> kubectl get svc -n argocd
NAME                                TYPE          CLUSTER-IP    EXTERNAL-IP    PORT(S)          AGE
argocd-applicationset-controller    ClusterIP     10.96.5.140    <none>         7000/TCP,8080/TCP 3d3h
argocd-dex-server                   ClusterIP     10.96.22.23    <none>         5556/TCP,5557/TCP,5558/TCP 3d3h
argocd-metrics                      ClusterIP     10.96.151.207  <none>         8082/TCP          3d3h
argocd-notifications-controller-metrics ClusterIP     10.96.131.97   <none>         9001/TCP          3d3h
argocd-redis                        ClusterIP     10.96.131.125  <none>         6379/TCP          3d3h
argocd-repo-server                  ClusterIP     10.96.133.73   <none>         8081/TCP,8084/TCP 3d3h
argocd-server                       ClusterIP     10.96.115.248  <none>         80/TCP,443/TCP    3d3h
argocd-server-metrics               ClusterIP     10.96.35.19    <none>         8083/TCP          3d3h

```

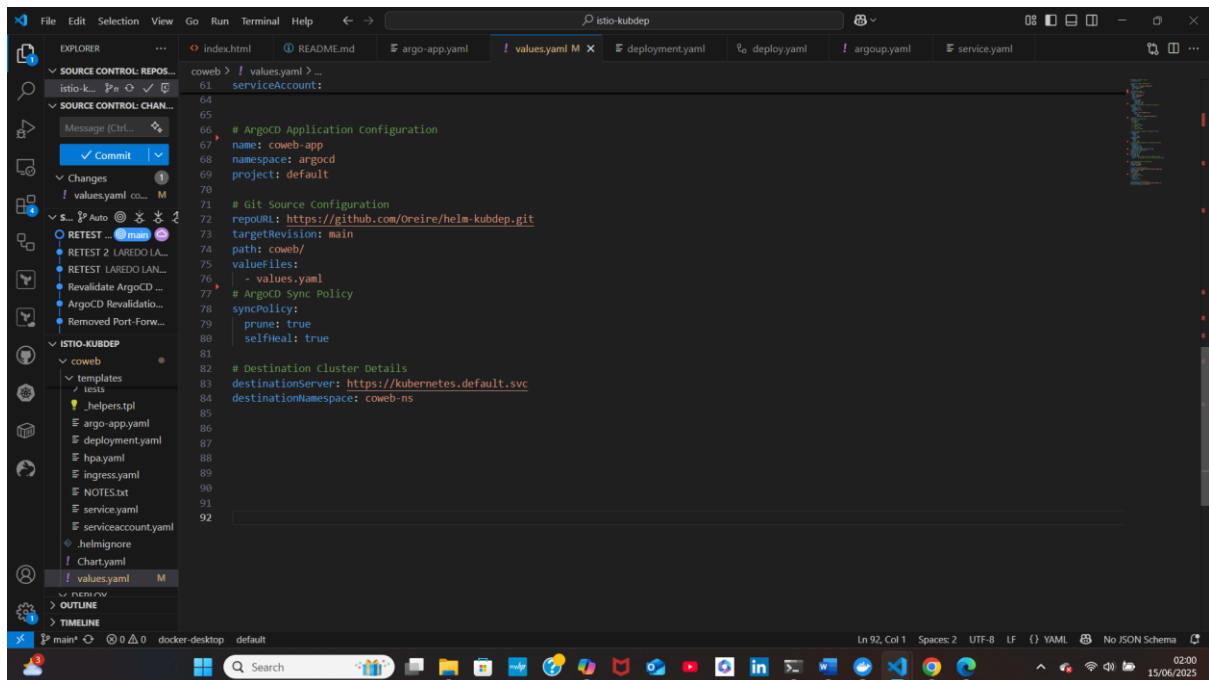
ArgoCD UI



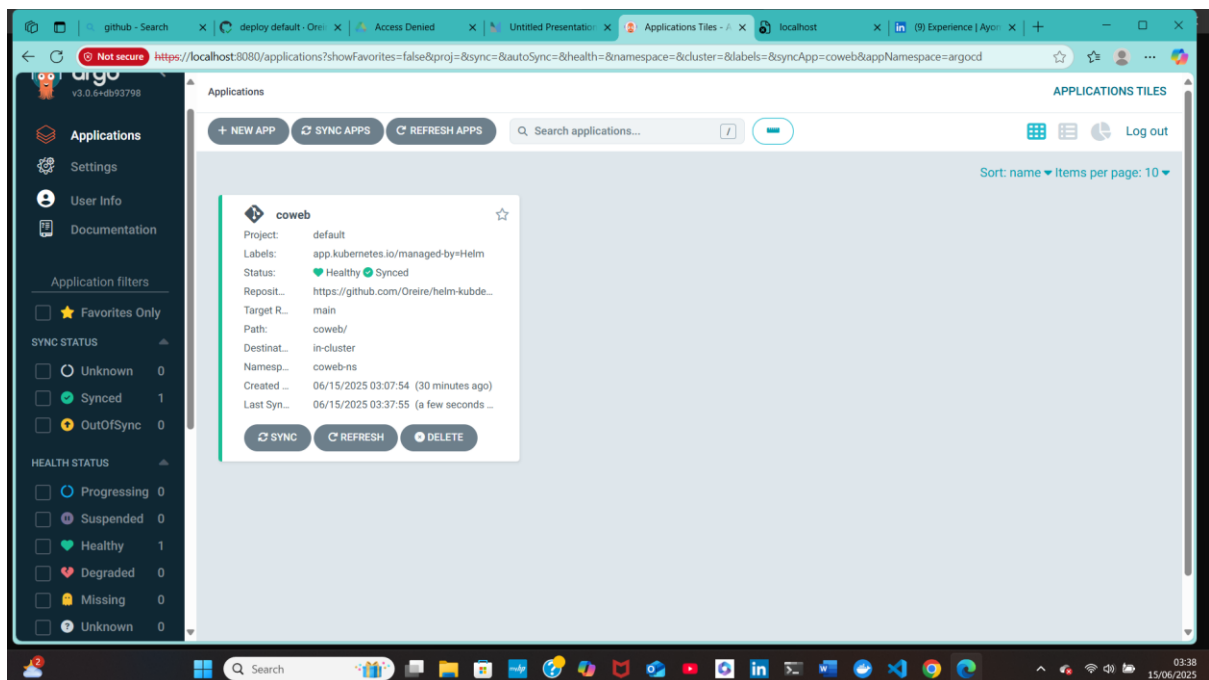
ArgoCD argo-app.yaml file



ArgoCD Helm values.yaml



ArgoCD Sync UI

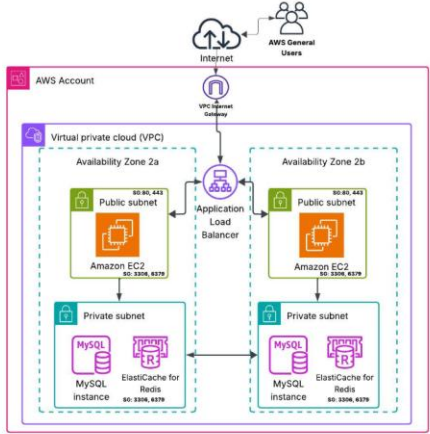


Web Application Deployed

localhost:31555

PROOF OF CONCEPT FOR KUBERNETES DEPLOYMENT USING GITHUB ACTIONS CICD AND HELM

This web application is designed to test the deployment of a Containerised Web Application Using GitHub Actions, ArgoCD and Istio Services Meshes. The HTML file would be updated progressively to evaluate the success of the project.



The diagram illustrates an AWS multi-availability zone architecture. It features a VPC with two public subnets (SO-BL-443) and two private subnets (SO-3004-8279) across Availability Zones 2a and 2b. An Application Load Balancer is positioned in the public subnets, routing traffic to Amazon EC2 instances (SO-3004-8279). The private subnets host MySQL instances and ElastiCache for Redis instances (SO-3004-8279). The architecture is connected to the Internet via an Internet Gateway and is accessible to AWS General Users.

localhost:31555

HIGHLIGHTS AND KEY CAPABILITIES

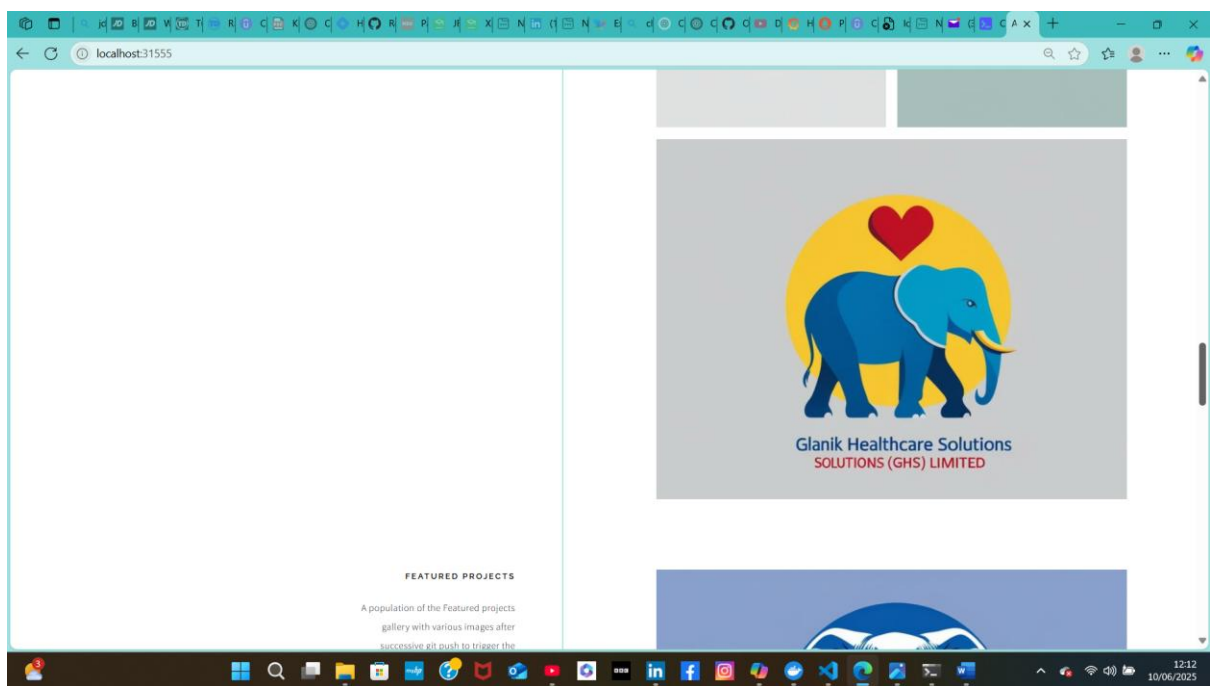
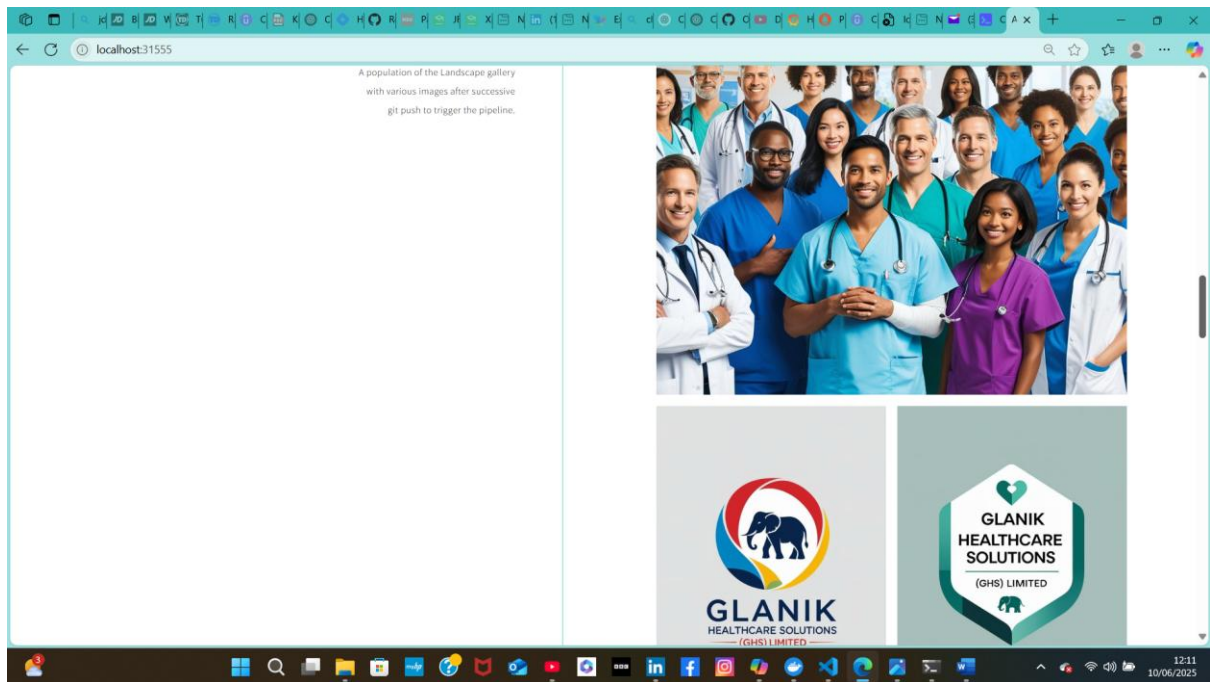
Workflow Tasks includes the functional tasks integrated with the GitHub Actions pipeline to enhance code semantics and security.

- Linting of Helm Chart
- Webhook Set-up
- Location-ready
- SonarQube Integrations
- Customizable
- Developer friendly

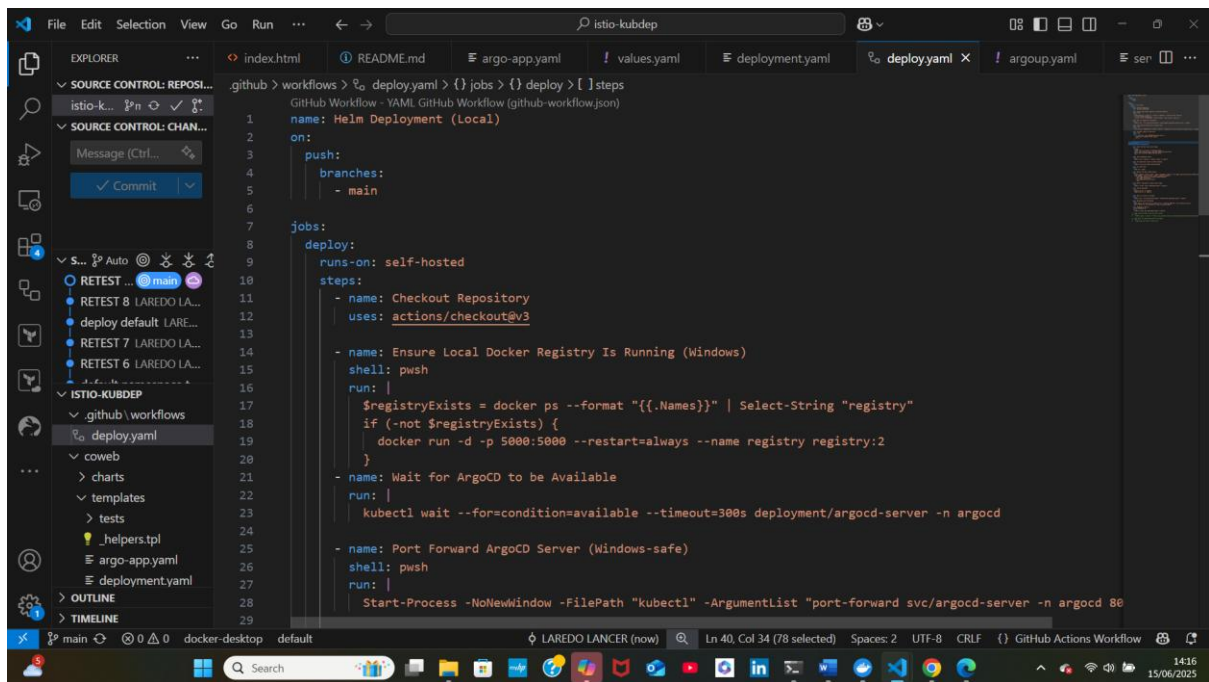
It's easy to update content, adjust layout styles, and include additional features using standard HTML and CSS.

IMAGE GALLERIES

Gallery Layouts was used to showcase the automatic deployment of the containerised web application after new image builds are triggered by the GitHub Actions CICD push.



GitHub Actions Workflow



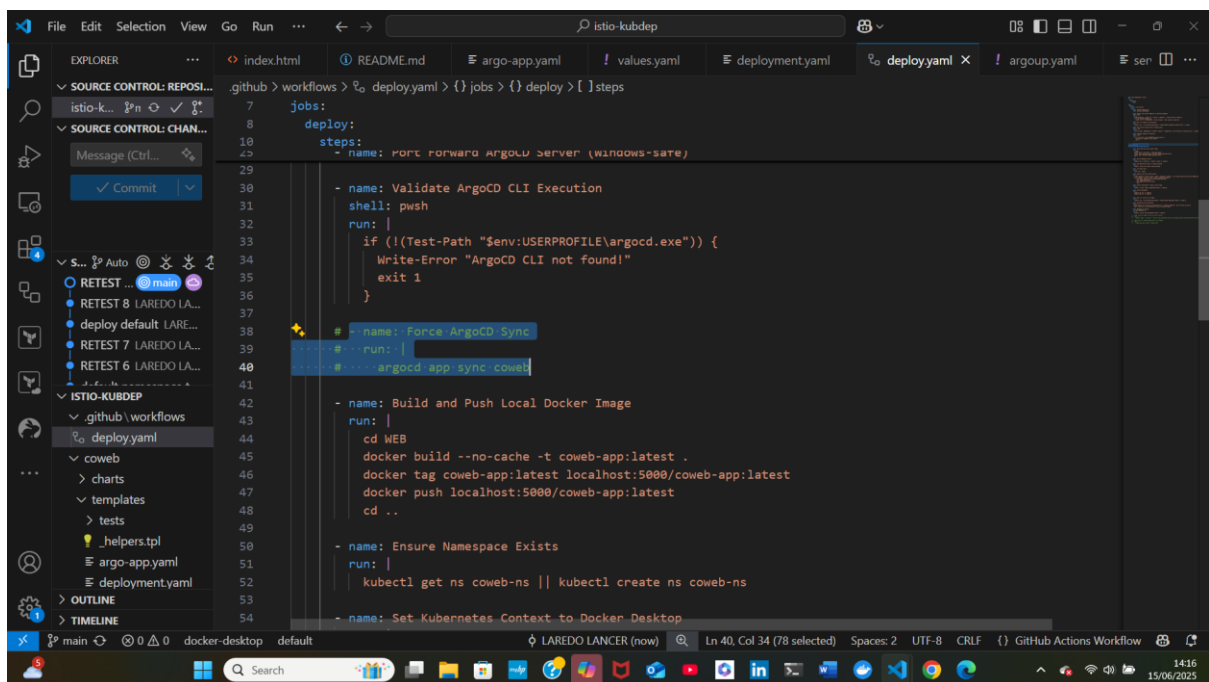
The screenshot shows the Visual Studio Code editor with a GitHub Actions workflow file named `deploy.yml` open. The file is located in the `.github/workflows` directory. The workflow is named `Helm Deployment (Local)` and is triggered on a `push` to the `main` branch. The workflow consists of a single job named `deploy` that runs on `self-hosted` runners. The job includes several steps: `Checkout Repository` (using `actions/checkout@v3`), `Ensure Local Docker Registry Is Running (Windows)` (using `pwsh` to run a script that checks for and starts a Docker registry), `Wait for ArgoCD to be Available` (using `kubectl wait` to ensure the `argocd-server` is available), and `Port Forward ArgoCD Server (Windows-safe)` (using `Start-Process` to run `kubectl port-forward`). The workflow is saved in the `deploy.yml` file.

```
name: Helm Deployment (Local)
on:
  push:
    branches:
      - main
jobs:
  deploy:
    runs-on: self-hosted
    steps:
      - name: Checkout Repository
        uses: actions/checkout@v3

      - name: Ensure Local Docker Registry Is Running (Windows)
        shell: pwsh
        run: |
          $registryExists = docker ps --format "{{.Names}}" | Select-String "registry"
          if (-not $registryExists) {
            docker run -d -p 5000:5000 --restart=always --name registry registry:2
          }

      - name: Wait for ArgoCD to be Available
        run: |
          kubectl wait --for=condition=available --timeout=300s deployment/argocd-server -n argocd

      - name: Port Forward ArgoCD Server (Windows-safe)
        shell: pwsh
        run: |
          Start-Process -NoNewWindow -FilePath "kubectl" -ArgumentList "port-forward svc/argocd-server -n argocd 80
```



The screenshot shows the Visual Studio Code editor with the same `deploy.yml` file. A new step has been added to the workflow, named `Force ArgoCD Sync`. This step is a `run` step that uses `pwsh` to run a script that checks for the presence of the `argocd.exe` file in the user's profile. If the file is not found, it writes an error message and exits with a status of 1. The step is highlighted with a yellow background. The workflow is saved in the `deploy.yml` file.

```
name: Helm Deployment (Local)
on:
  push:
    branches:
      - main
jobs:
  deploy:
    runs-on: self-hosted
    steps:
      - name: Checkout Repository
        uses: actions/checkout@v3

      - name: Ensure Local Docker Registry Is Running (Windows)
        shell: pwsh
        run: |
          $registryExists = docker ps --format "{{.Names}}" | Select-String "registry"
          if (-not $registryExists) {
            docker run -d -p 5000:5000 --restart=always --name registry registry:2
          }

      - name: Wait for ArgoCD to be Available
        run: |
          kubectl wait --for=condition=available --timeout=300s deployment/argocd-server -n argocd

      - name: Port Forward ArgoCD Server (Windows-safe)
        shell: pwsh
        run: |
          Start-Process -NoNewWindow -FilePath "kubectl" -ArgumentList "port-forward svc/argocd-server -n argocd 80

      - name: Force ArgoCD Sync
        run: |
          pwsh -c {
            if (!(Test-Path "$env:USERPROFILE\argocd.exe")) {
              Write-Error "ArgoCD CLI not found!"
              exit 1
            }
          }

      - name: Build and Push Local Docker Image
        run: |
          cd WEB
          docker build --no-cache -t coweb-app:latest .
          docker tag coweb-app:latest localhost:5000/coweb-app:latest
          docker push localhost:5000/coweb-app:latest
          cd ..

      - name: Ensure Namespace Exists
        run: |
          kubectl get ns coweb-ns || kubectl create ns coweb-ns

      - name: Set Kubernetes Context to Docker Desktop
```


This screenshot shows a Visual Studio Code editor with a GitHub Actions workflow file named `deploy.yml` open. The workflow is configured to run on a `ubuntu-latest` runner. It defines a single job named `deploy` with the following steps:

- Set Kubernetes Context to Docker Desktop:** Uses `kubectl config use-context docker-desktop` to set the context.
- Lint Helm Chart:** Uses `helm lint ./coweb` to validate the chart.
- Deploy with Helm (Stable Setup):** Uses `helm upgrade --install coweb ./coweb --namespace coweb-ns --set image.repository=localhost:5000/coweb-app --set image.tag=latest --set image.pullPolicy=Always --wait` to deploy the application.
- Restart Deployment to Apply Latest Image:** Uses `kubectl rollout restart deployment/coweb -n coweb-ns` to restart the deployment.
- Verify Deployment:** Uses `kubectl get all -n coweb-ns` and `kubectl get svc -n coweb-ns` to verify the deployment.

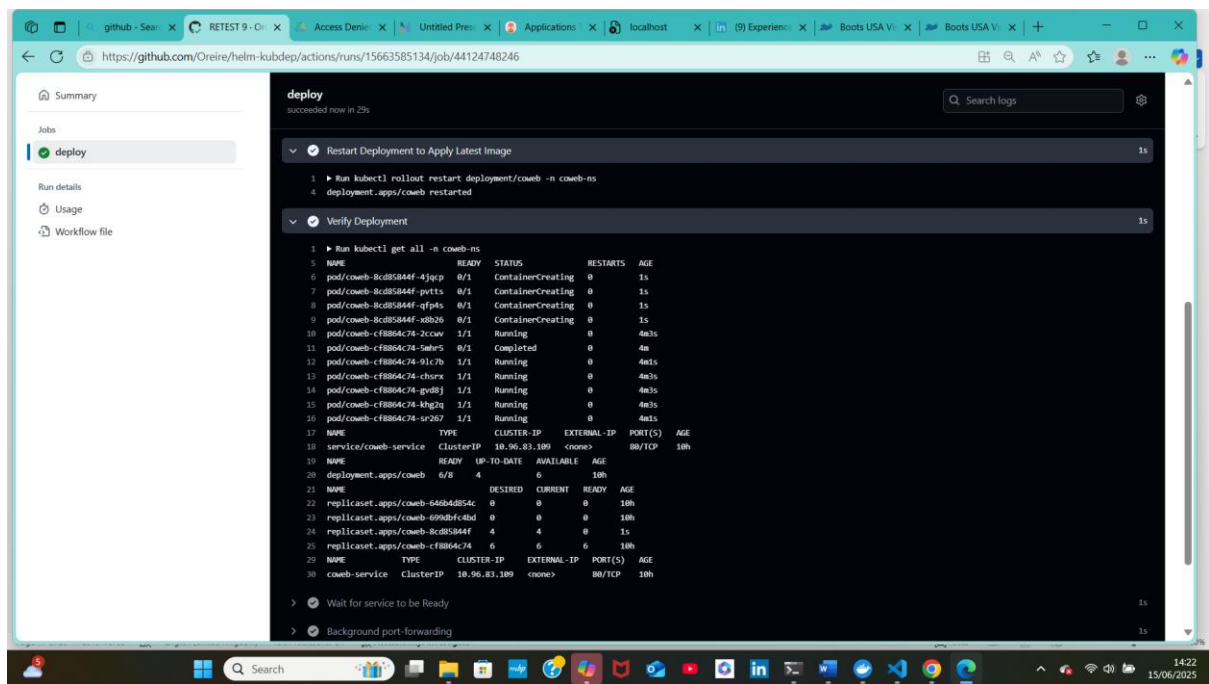
The Explorer sidebar on the left shows the project structure, including `github/workflows` and `coweb` directories. The bottom status bar indicates the file is part of a GitHub Actions Workflow.

This screenshot shows a Visual Studio Code editor with a GitHub Actions workflow file named `deploy.yml` open. The workflow is configured to run on a `ubuntu-latest` runner. It defines a single job named `deploy` with the following steps:

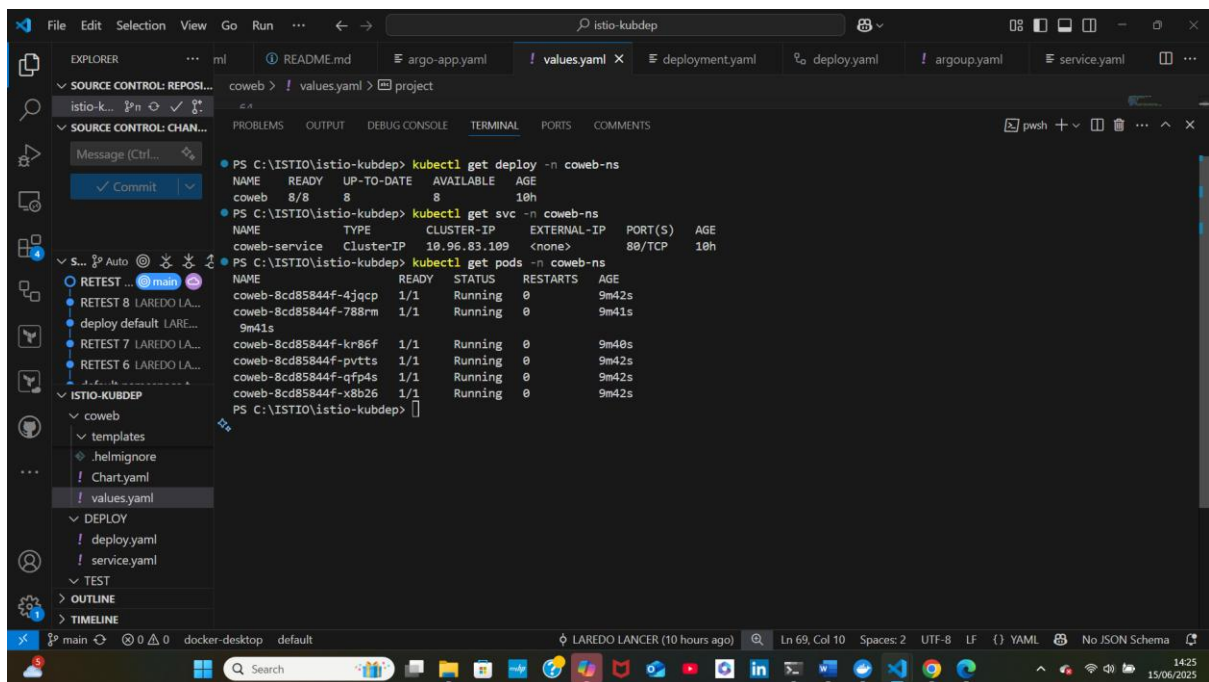
- Wait for service to be Ready:** Uses `kubectl wait --for=condition=available --timeout=150s deployment/coweb -n coweb-ns` to wait for the deployment to be ready.
- Background port-forwarding:** Uses `nohup kubectl port-forward svc/coweb-service -n coweb-ns 30015:80 > port-forward.log 2>&1 & echo "Service is now available at http://localhost:30015"` to forward traffic.
- Rollback on Failure:** Uses `kubectl rollout undo deployment/coweb -n coweb-ns` to rollback the deployment if it fails.
- Install ArgoCD in the local k8s cluster:** Uses `kubectl apply -n argocd -f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install` to install ArgoCD.
- Wait for ArgoCD Application to be Ready:** Uses `argocd app wait coweb --timeout 300` to wait for the application to be ready.

The Explorer sidebar on the left shows the project structure, including `github/workflows` and `coweb` directories. The bottom status bar indicates the file is part of a GitHub Actions Workflow.

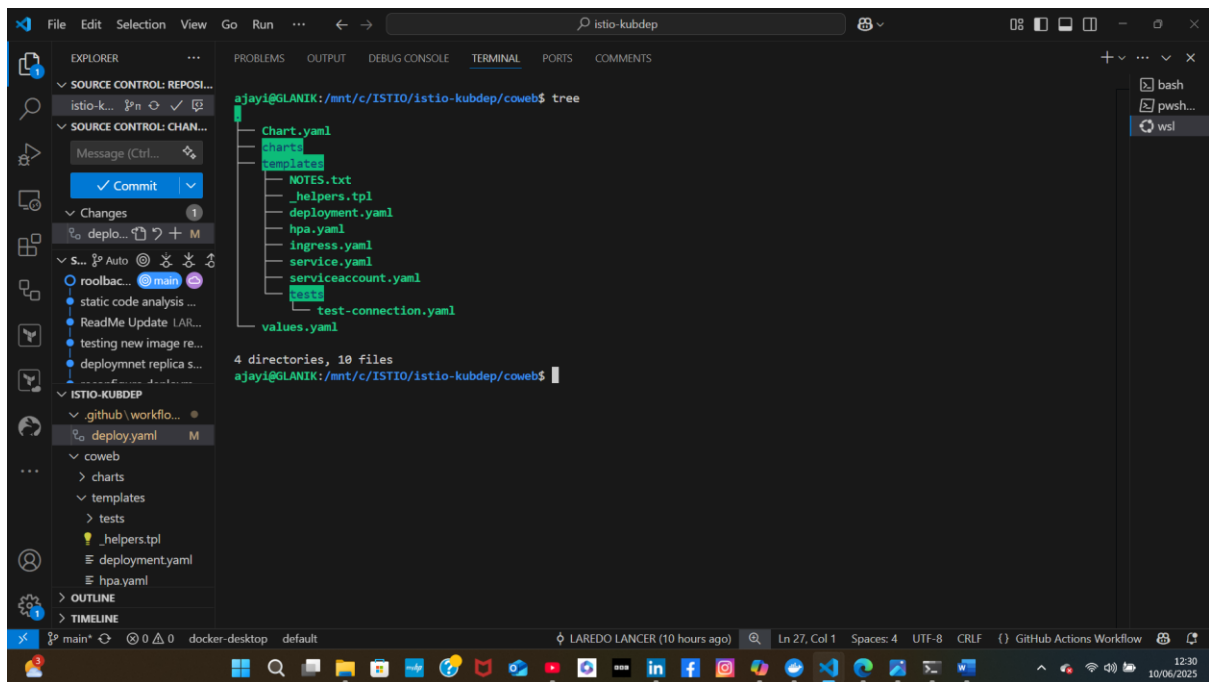
GitHub Actions Workflow Success



Deployment, service and pods



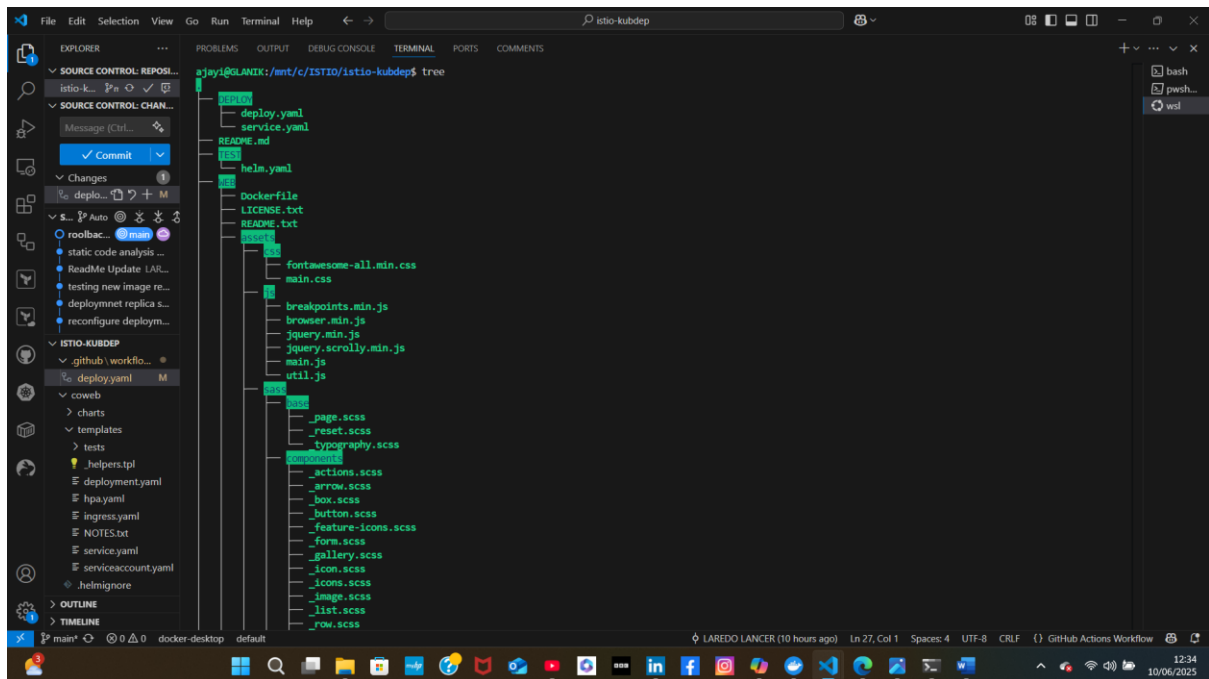
Helm Chart Tree



The screenshot shows the Visual Studio Code interface with the Explorer view on the left and the File Explorer view in the center. The Explorer view shows the file structure of the 'istio-kubdep' chart, which includes a 'charts' directory, a 'templates' directory, and a 'tests' directory. The File Explorer view shows the contents of the 'istio-kubdep' chart, which includes a 'Chart.yaml' file, a 'values.yaml' file, and a 'test-connection.yaml' file. The terminal at the bottom shows the command 'tree' being executed, displaying the directory structure of the chart.

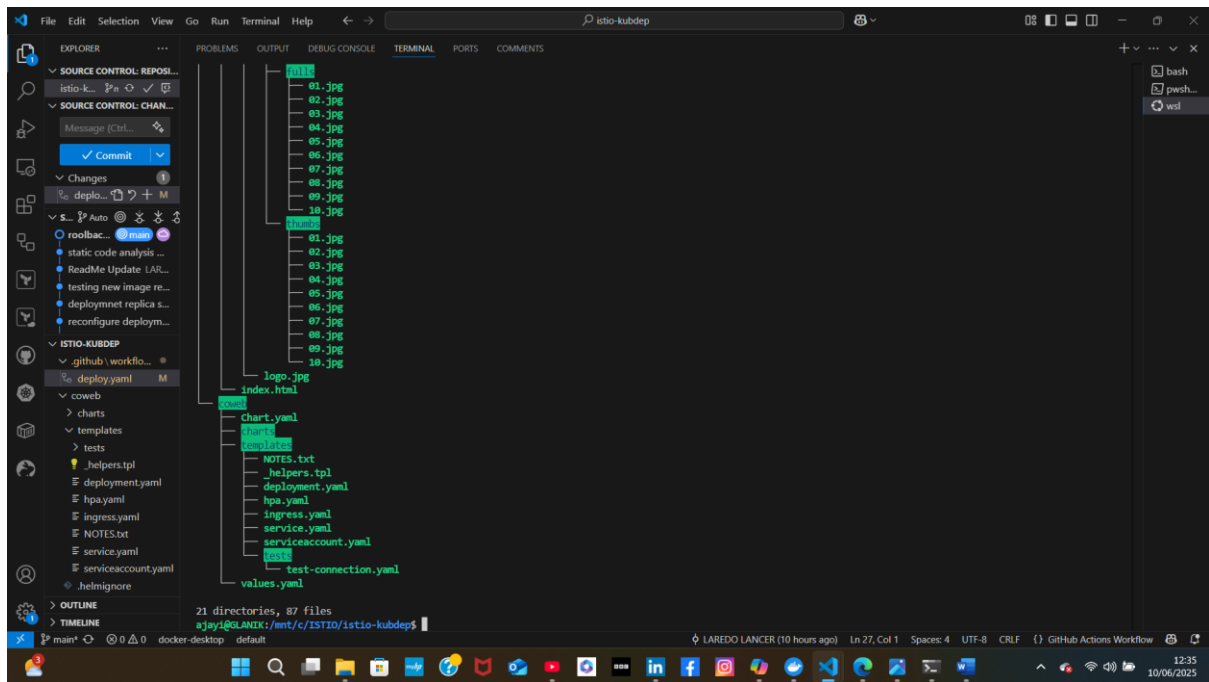
```
ajayi@GLANIK:/mnt/c/ISTIO/istio-kubdep/oweb$ tree
.
├── Chart.yaml
├── templates
│   ├── NOTES.txt
│   ├── _helpers.tpl
│   ├── deployment.yaml
│   ├── hpa.yaml
│   ├── ingress.yaml
│   ├── service.yaml
│   ├── serviceaccount.yaml
│   └── test-connection.yaml
└── values.yaml

4 directories, 10 files
ajayi@GLANIK:/mnt/c/ISTIO/istio-kubdep/oweb$
```

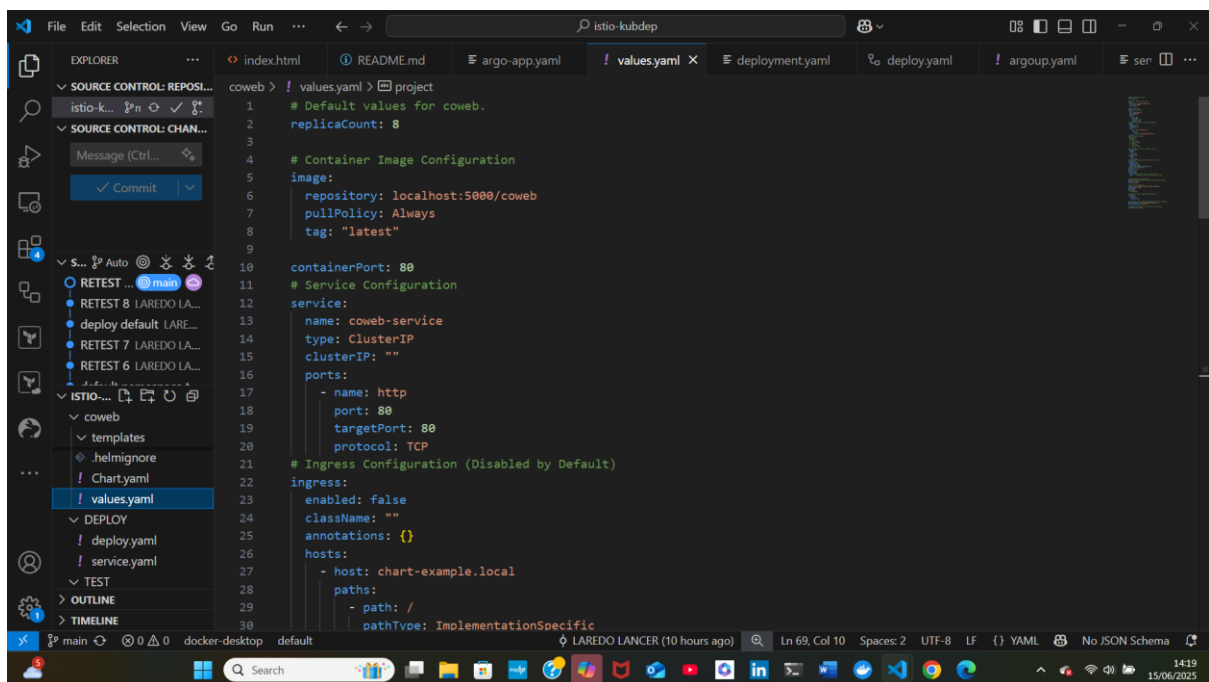


The screenshot shows the Visual Studio Code interface with the Explorer view on the left and the File Explorer view in the center. The Explorer view shows the file structure of the 'istio-kubdep' chart, which includes a 'charts' directory, a 'templates' directory, and a 'tests' directory. The File Explorer view shows the contents of the 'istio-kubdep' chart, which includes a 'Chart.yaml' file, a 'values.yaml' file, and a 'test-connection.yaml' file. The terminal at the bottom shows the command 'tree' being executed, displaying the directory structure of the chart.

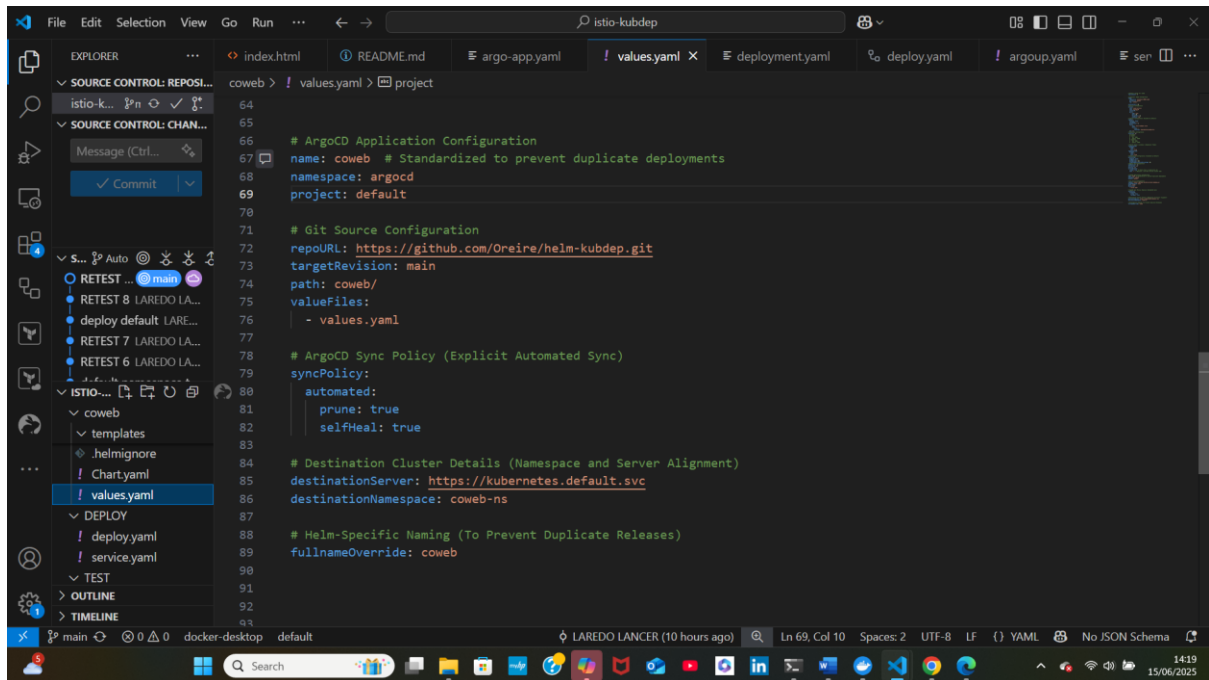
```
ajayi@GLANIK:/mnt/c/ISTIO/istio-kubdep$ tree
.
├── deploy.yaml
├── service.yaml
├── README.md
├── helm.yaml
├── Dockerfile
├── LICENSE.txt
├── README.txt
├── test-connection.yaml
├── fontawesome-all.min.css
├── main.css
├── breakpoints.min.js
├── browser.min.js
├── jquery.min.js
├── jquery.scroll.min.js
├── main.js
├── util.js
├── _page.scss
├── _reset.scss
├── typography.scss
├── _actions.scss
├── _arrow.scss
├── _box.scss
├── _button.scss
├── _feature-icons.scss
├── _form.scss
├── _gallery.scss
├── _icon.scss
├── _icons.scss
├── _image.scss
├── _list.scss
├── _row.scss
```



Helm coweb-service (Values.yaml)



ArgoCD Helm values.yaml



```
64
65
66 # ArgoCD Application Configuration
67 name: coweb # Standardized to prevent duplicate deployments
68 namespace: argocd
69 project: default
70
71 # Git Source Configuration
72 repoURL: https://github.com/Oreire/helm-kubdep.git
73 targetRevision: main
74 path: coweb/
75 valueFiles:
76   - values.yaml
77
78 # ArgoCD Sync Policy (Explicit Automated Sync)
79 syncPolicy:
80   automated:
81     prune: true
82     selfHeal: true
83
84 # Destination Cluster Details (Namespace and Server Alignment)
85 destinationServer: https://kubernetes.default.svc
86 destinationNamespace: coweb-ns
87
88 # Helm-Specific Naming (To Prevent Duplicate Releases)
89 fullnameOverride: coweb
90
91
92
93
```

6.0 Conclusion

Organisations deploying containerised web applications on Kubernetes often rely on tools like GitHub Actions, CodeRabbit, Helm, and ArgoCD to enhance automation, efficiency, and governance across their development and deployment workflows. **GitHub Actions** streamlines CI/CD pipelines by automating builds, tests, and deployments triggered by code events. Moreover, its integration with repositories and support for reusable workflows ensure consistent and reproducible processes across environments.

Building on this automation layer, CodeRabbit introduces AI-powered code reviews that help maintain high code quality and accelerate development. By identifying potential issues and vulnerabilities early, it enhances application stability while enforcing best practices and strengthening compliance efforts.

Meanwhile, Helm simplifies Kubernetes application management through standardised, reusable charts, ensuring efficient deployment and configuration. Additionally, it facilitates scalable operations by enabling version control, parameterised deployments, and seamless rollbacks, improving reliability across different stages and environments.

To complete the pipeline, ArgoCD leverages a GitOps-based approach to deployment management, maintaining the desired state of applications in Git. As a result, it enables declarative, consistent, and auditable control over deployments while minimising configuration drift and supporting multi-cluster scalability.

Together, these tools create a fully integrated DevOps ecosystem that reduces manual intervention, increases release velocity, and ensures resilient, secure, and scalable delivery of modern web applications on Kubernetes.

7.0 Next-Steps

To optimize this CI/CD workflow for production readiness, the following enhancements are under considerations:

- **Integrate DevSecOps practices** to strengthen security and manage secrets more effectively throughout the pipeline.
- **Introduce automated testing stages**, including unit, integration, and Kubernetes smoke tests, to validate application behavior prior to deployment.
- **Implement monitoring and observability** using Istio to gain insight into service performance, traffic flow, and runtime anomalies.